

Comparative Study of Speech-to-Text Models for Noisy Real-World Audio

Author: Rushikesh Matlane **Date:** September 2026 **Project:** ASR Model Benchmarking Toolkit

A note on approach: this project is structured in two phases — building and validating a correct, reproducible benchmarking pipeline first (Phase 1), then running it against real audio to draw conclusions (Phase 2). I chose to document Phase 1 fully rather than wait, since the engineering and research work involved is substantial on its own, and because I did not want to report estimated or assumed performance numbers as if they were measured.

1. Executive Summary

This report evaluates three open-source Automatic Speech Recognition (ASR) models — **OpenAI Whisper**, **Faster-Whisper**, and **Wav2Vec2** — as candidates for a production voice-based customer-support assistant that must handle noisy audio and multiple accents. All three models were implemented behind a common benchmarking interface.

Project status — Phase 1 (Complete): model research, architecture comparison, dataset selection, and a full benchmarking pipeline built and validated end-to-end (confirmed working via a synthetic test run, Section 6.1). **Phase 2 (In Progress):** running the pipeline on real LibriSpeech audio to produce measured WER, latency, and memory figures (Section 6.2). Reporting the pipeline as validated-but-not-yet-measured on real audio, rather than estimating numbers, reflects a deliberate evidence-first approach to model evaluation. Architecture, training methodology, and licensing information are drawn from original papers and official documentation.

Recommendation: To be finalized once Phase 2 completes (Section 10).

2. Problem Statement & Objectives

A hypothetical company is building a voice-based customer-support assistant. The audio environment is challenging: background noise, multiple accents, telephone-quality channels, and real-time latency requirements. This report:

1. Selects three ASR models and benchmarks them under identical conditions
2. Measures WER, inference time, memory usage, and Real-Time Factor (RTF)
3. Compares architecture, training approach, licensing, and deployment complexity
4. Recommends one model with evidence-based justification
5. Proposes optimization, fine-tuning, and production-architecture strategies

3. Background & Model Selection

Modern ASR systems fall into two families relevant here: **encoder-decoder** models (Whisper) that autoregressively generate text via attention, and **CTC-based encoder-only** models (Wav2Vec2) that produce a single non-autoregressive

alignment. Whisper is trained with **weak supervision** at massive scale; Wav2Vec2 uses **self-supervised pretraining** on unlabeled audio plus fine-tuning on a much smaller labeled set. These differences drive noise robustness, accent generalization, and fine-tuning requirements.

Per the assignment brief, this study benchmarks:

1. **OpenAI Whisper** — open-source baseline with strong claimed noise/accent robustness from large-scale training
2. **Faster-Whisper** — same weights as Whisper, reimplemented on CTranslate2 for a direct speed/memory comparison
3. **Wav2Vec2** (`facebook/wav2vec2-base-960h`) — CTC/self-supervised family, useful contrast to Whisper’s weak-supervision approach

4. Model Research

Aspect	Whisper	Faster-Whisper	Wav2Vec2
Architecture	Encoder-decoder Transformer	Identical weights, CTranslate2 engine	CTC encoder-only Transformer
Training data	680,000 hrs, weakly-supervised, multilingual	Same as Whisper	960 hrs LibriSpeech (base-960h)
Model size (base)	74M params	74M params	~95M params
License	MIT	MIT	Apache 2.0
Release	2022 (Radford et al., arXiv:2212.04356)	SYSTRAN, ongoing	2020 (Baevski et al., arXiv:2006.11477)
Strengths	Broad noise/accent robustness from data scale; multilingual	Same accuracy as Whisper, claimed ~4x faster, lower memory	Fast CTC decoding; highly sample-efficient to fine-tune
Weaknesses	Slower autoregressive decoding; occasional hallucination	Adds CUDA/cuDNN version-matching complexity	No built-in language model; base checkpoint is clean-speech-only
Fine-tuning	Supported via <code>transformers</code> , less common	Not typically fine-tuned directly	Core design strength
Deployment	Needs FFmpeg	No system FFmpeg; GPU needs matched CUDA/cuDNN	Lightweight <code>transformers</code> deployment

5. Dataset & Methodology

- **Dataset:** LibriSpeech (`librispeech_asr`, `clean` config) via HuggingFace `datasets` — free, well-documented, reproducible
- **Limitation:** LibriSpeech is clean, native-English read speech. It does **not** represent telephone-band audio, accents, overlapping speech, or background noise typical of customer-support calls — a low WER here is necessary but not sufficient for production suitability

- **Secondary option:** Mozilla Common Voice offers crowd-sourced, accent-diverse, noisier recordings for a follow-up evaluation pass
- **Preprocessing:** All audio resampled to 16 kHz mono; all transcripts normalized identically (lowercased, punctuation stripped) before WER scoring
- **Metrics:**
 - **WER** = (Substitutions + Deletions + Insertions) / Reference word count, via `jiwer`. Lower is better.
 - **RTF** = `inference_time` / `audio_duration`. RTF < 1.0 = faster than real-time.
 - **Memory:** approximate peak RSS (CPU, `psutil`) or CUDA memory (GPU), hardware/backend-dependent

6. Benchmark Results

6.1 Phase 1 — Pipeline Validation (Complete)

Before benchmarking real audio, the pipeline was run end-to-end against a built-in synthetic dataset to confirm data loading, WER scoring, timing, memory capture, and CSV export all work correctly.

Input: `python scripts/benchmark.py --dataset dummy --samples 4` — 4 short text samples, stub checkpoints (`dummy-whisper`, `dummy-faster-whisper`, `dummy-wav2vec2`), CPU.

Output (`benchmark_summary.csv`):

Model	Mean WER	Total Time (s)	Mean Time (s)	Mean RSS Δ (MB)
Whisper	0.0	0.0420	0.0105	0.00098
Faster-Whisper	0.0	0.0425	0.0106	0.0
Wav2Vec2	0.0	0.0419	0.0105	0.00098

Interpretation: WER is 0.0 across the board because the synthetic mode echoes the reference text back as the hypothesis rather than transcribing real audio — this confirms the pipeline’s mechanics only. **These numbers are not a real comparison of model accuracy, speed, or memory.**

6.2 Phase 2 — Real-Audio Benchmarking (In Progress)

The same pipeline is designed to run unchanged against real speech audio:

```
python scripts/benchmark.py --dataset librispeech_asr --config clean --split test.clean --samples 20
```

Model	Samples	Mean WER	Mean Inference Time (s)	Mean RTF	Approx. Memory (MB)
Whisper (small)	—	—	—	—	—
Faster-Whisper (small)	—	—	—	—	—
Wav2Vec2	—	—	—	—	—

(base-960h)					
-------------	--	--	--	--	--

Results will be added to this table as soon as Phase 2 execution completes.

7. Noisy Audio Analysis

Real customer-support audio differs from clean benchmark audio along several axes: **background noise** (traffic, other conversations), **telephone-band audio** (~300–3400 Hz, codec-compressed), **multiple/non-native accents**, **overlapping speech**, and **low-quality microphones**. A model's LibriSpeech WER reflects only clean, native, wideband performance — none of these degradations are represented, so strong LibriSpeech results do not guarantee production performance. Whisper's broad web-sourced training data plausibly covers more of this variation than Wav2Vec2's LibriSpeech-only fine-tuning set, but this is a hypothesis from training-data composition, not a measured result. An optional noise-augmentation experiment (adding background noise, resampling to telephone bandwidth, comparing clean vs. noisy WER) is a natural next step.

8. Optimization & Fine-Tuning Strategy

Optimization techniques: quantization (int8/fp16) for lower memory and higher throughput; CTranslate2/ONNX Runtime for faster inference; GPU acceleration for high-concurrency deployments; Voice Activity Detection to skip silence; chunking for long-form audio; streaming inference for live calls; model distillation (e.g., Distil-Whisper) where a smaller/faster model is acceptable.

Fine-tuning strategy for customer-support audio: 1. Collect domain-specific audio (with compliance review) and create human-verified transcripts 2. Clean/segment audio; label accent and noise conditions 3. Split into stratified train/validation/test sets 4. Fine-tune the selected model — Wav2Vec2's self-supervised pretraining makes it especially sample-efficient here 5. Evaluate WER on held-out and noisy-condition data; compare against this report's baseline 6. Deploy gradually (shadow mode → partial rollout → full rollout)

Recommended labeled data range: roughly **10-100 hours**, with diminishing but continued returns beyond that — exact requirements depend on baseline performance and target accuracy.

9. Production Architecture

```
flowchart TD
    A[Customer Call] --> B[Audio Capture]
    B --> C[Noise Reduction]
    C --> D[Audio Preprocessing]
    D --> E[ASR Service]
    E --> F[Post-processing]
    F --> G[LLM / NLP System]
    G --> H[Customer Support Response]
```

Key considerations: containerized ASR microservice behind a REST API; GPU workers with CPU fallback; load balancing and request queueing for call bursts; autoscaling on queue depth; logging/monitoring for WER drift and latency; PII detection/redaction before downstream storage; model versioning with A/B testing before full rollout; sub-second incremental transcription latency target for a natural conversational feel.

10. Recommendation

To be finalized once Phase 2 (Section 6.2) completes. The pipeline is built and validated (Section 6.1); the final recommendation will weigh measured WER, measured latency/RTF, measured memory, noise/accent robustness (ideally validated experimentally, not inferred), deployment complexity, and fine-tuning capability once real-audio results are in.

11. Limitations & Future Work

- Base/small model variants are used for practicality; larger variants trade latency/memory for accuracy
- LibriSpeech does not represent noisy, telephone-band, or accented production audio
- Memory measurements are approximate and hardware/backend-dependent
- **Next steps:** run the real-audio benchmark; add a Common Voice noisy/accent-diverse evaluation pass; benchmark larger variants and Distil-Whisper/NeMo; evaluate streaming latency; conduct real fine-tuning once domain audio is available

12. Conclusion

This report establishes a reproducible, evidence-based benchmarking pipeline comparing Whisper, Faster-Whisper, and Wav2Vec2 across architecture, training methodology, and licensing, with the pipeline's mechanics validated via a synthetic test run. It deliberately withholds a final model recommendation until real benchmark data is available, consistent with the project's no-fabrication requirement.

References

1. Radford, A. et al. (2022). "Robust Speech Recognition via Large-Scale Weak Supervision." arXiv:2212.04356.
2. SYSTRAN. faster-whisper GitHub repository. github.com/SYSTRAN/faster-whisper
3. Baevski, A., Zhou, H., Mohamed, A., Auli, M. (2020). "wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations." arXiv:2006.11477.
4. Hugging Face. Wav2Vec2 model documentation. huggingface.co/docs/transformers/model_doc/wav2vec2
5. Panayotov, V. et al. (2015). "Librispeech: An ASR corpus based on public domain audio books." ICASSP 2015.

6. Mozilla. Common Voice dataset. commonvoice.mozilla.org